Robot Kinematics Calculation with Maple

> **Notice:** The runnable Maple scripts (`FK-and-IK.mw`, `jacobian.mw`) are attached in the folder.

I. Transformation Matrices and Analytical Inverse Kinematics

Part 1: Extracted Kinematic Parameters

From the URDF, we extract the following link lengths and offsets (all coordinates are in meters):

- **Height of Joint 1 base:** $h = 0.2435\,\mathrm{m}$ (from base Z offset)
- **Length of Link 2:** $a_2 = 0.2002\,\mathrm{m}$ (from Joint 3 X offset)
- **X-offset of Joint 4:** $a_3 = 0.087\,\mathrm{m}$ (from Joint 4 X offset)
- **Y-offset of Joint 4 (under-arm twist):** $d_4 = 0.22761\,\mathrm{m}$ (from Joint 4 Y offset of -0.22761)
- **Y-offset of Joint 6 (wrist length):** $d_6 = 0.0625\,\mathrm{m}$ (from Joint 6 Y offset)

Part 2: Relative Transformations $T_i^{i-1}(q_i)$

Each joint transformation is constructed using the URDF `<origin>` roll-pitch-yaw (using the standard ZYX extrinsic Euler convention $R = R_z(\mathrm{yaw})R_y(\mathrm{pitch})R_x(\mathrm{roll})$) followed by a rotation of q_i about the joint's Z-axis.

Relative transformations:

$$T_1^0(q_1) = \begin{bmatrix} \cos(q_1) & -\sin(q_1) & 0 & 0 \\ \sin(q_1) & \cos(q_1) & 0 & 0 \\ 0 & 0 & 1 & h \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_2^1(q_2) = \begin{bmatrix} \sin(q_2) & \cos(q_2) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \cos(q_2) & -\sin(q_2) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_3^2(q_3) = \begin{bmatrix} \sin(q_3) & \cos(q_3) & 0 & a_2 \\ \cos(q_3) & -\sin(q_3) & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_4^3(q_4) = \begin{bmatrix} \cos(q_4) & -\sin(q_4) & 0 & a_3 \\ 0 & 0 & -1 & -d_4 \\ \sin(q_4) & \cos(q_4) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_5^4(q_5) = \begin{bmatrix} \cos(q_5) & -\sin(q_5) & 0 & 0 \\ 0 & 0 & -1 & 0 \\ \sin(q_5) & \cos(q_5) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_6^5(q_6) = \begin{bmatrix} \cos(q_6) & -\sin(q_6) & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ -\sin(q_6) & -\cos(q_6) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Part 3: Proof of Spherical Wrist & Kinematic Decoupling

To use analytical decoupled inverse kinematics, the axes of Joints 4, 5, and 6 must intersect at a single point (the **wrist center**, P_{wc}).

1. The origin of Frame 4 is coincident with the origin of Frame 5 (due to the zero-translation vector in T_5^4). Therefore, the axes of Joint 4 (Z_4) and Joint 5 (Z_5) intersect at this point.
2. The translation of Joint 6 relative to Frame 5 is $[0, d_6, 0]^T$ (along Y_5).
3. The rotational axis of Joint 6 (Z_6) is also pointing along Y_5 in the zero configuration ($q_6 = 0$). Since the rotational axis points directly along the line linking the origins O_5 and O_6, the line of axis Z_6 passes directly through the origin of Frame 5 ($O_5 = O_4$).

Because all three axes intersect at the origin of Frame 4, the robot has a spherical wrist.

Part 4: Analytical Solutions for Inverse Kinematics

Step 1: Compute Wrist Center from Target Pose

Given your target End-Effector transformation matrix T_{target} containing orientation R_{target} and position P_{target}:

$$T_{target} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & X_{target} \\ R_{21} & R_{22} & R_{23} & Y_{target} \\ R_{31} & R_{32} & R_{33} & Z_{target} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The wrist center is located at a distance of d_6 backwards along the local Z-axis of Joint 6:

$$P_{wc} = P_{target} - d_6 \cdot R_{target} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Expressing the coordinates:

$$P_{wc,x} = X_{target} - d_6 \cdot R_{13}$$

$$P_{wc,y} = Y_{target} - d_6 \cdot R_{23}$$

$$P_{wc,z} = Z_{target} - d_6 \cdot R_{33}$$

Step 2: Solve for Joint 1

$$q_1 = \text{atan2}(P_{wc,y}, P_{wc,x}) \quad \text{or} \quad q_1 = \text{atan2}(-P_{wc,y}, -P_{wc,x})$$

Using q_1, transform the wrist center into the base of Link 1:

$$v_{1,x} = P_{wc,x}\cos(q_1) + P_{wc,y}\sin(q_1)$$

$$v_{1,z} = P_{wc,z} - h$$

Step 3: Solve for Joint 2

We define a constant K:

$$K = \frac{v_{1,x}^2 + v_{1,z}^2 + a_2^2 - a_3^2 - d_4^2}{2a_2}$$

This reduces the system to $v_{1,x}\sin(q_2) + v_{1,z}\cos(q_2) = K$. The algebraic solution for q_2 is:

$$q_2 = \text{atan2}\left(K, \pm\sqrt{v_{1,x}^2 + v_{1,z}^2 - K^2}\right) - \text{atan2}(v_{1,z}, v_{1,x})$$

Step 4: Solve for Joint 3

We define the combined angle $\theta_{23} = q_2 - q_3$. Solve for θ_{23} via:

$$\cos(\theta_{23}) = \frac{a_3(v_{1,x} - a_2\sin(q_2)) - d_4(v_{1,z} - a_2\cos(q_2))}{a_3^2 + d_4^2}$$

$$\sin(\theta_{23}) = \frac{-d_4(v_{1,x} - a_2\sin(q_2)) - a_3(v_{1,z} - a_2\cos(q_2))}{a_3^2 + d_4^2}$$

$$\theta_{23} = \text{atan2}\left(\sin(\theta_{23}), \cos(\theta_{23})\right)$$

$$q_3 = q_2 - \theta_{23}$$

Step 5: Solve for Joints 4, 5, and 6

Compute the remaining orientation required by the wrist:

$$R_{\text{wrist}} = \left[R_1^0(q_1)R_2^1(q_2)R_3^2(q_3)\right]^T \cdot R_{target} = \begin{bmatrix} R'_{11} & R'_{12} & R'_{13} \\ R'_{21} & R'_{22} & R'_{23} \\ R'_{31} & R'_{32} & R'_{33} \end{bmatrix}$$

Extract the joint variables from the symbolic target elements:

$$q_5 = \text{atan2}\left(\pm\sqrt{1 - (R'_{23})^2}, -R'_{23}\right)$$

Assuming $\sin(q_5) \neq 0$:

$$q_4 = \text{atan2}\left(-R'_{33} \cdot \text{sgn}(\sin(q_5)), -R'_{13} \cdot \text{sgn}(\sin(q_5))\right)$$

$$q_6 = \text{atan2}\left(R'_{22} \cdot \text{sgn}(\sin(q_5)), -R'_{21} \cdot \text{sgn}(\sin(q_5))\right)$$

When $\sin(q_5) = 0$ (Wrist Singularity), axes Z_4 and Z_6 become collinear, leading to an infinite number of solutions for q_4 and q_6. In the C++ implementation, this is resolved by convention: q_4 is locked to its current physical joint angle ($q_{4,\text{current}}$), and q_6 absorbs the remaining required rotation.

Part 5: Maple Script for Verifying Kinematics Matrices

This Maple script varifies the forward kinematics transformations.

```
restart:
with(LinearAlgebra):

h := 2435/10000;
a2 := 2002/10000;
a3 := 87/1000;
d4 := 22761/100000;
d6 := 625/10000;

RotZ := q -> Matrix([[cos(q), -sin(q), 0, 0], [sin(q), cos(q), 0, 0], [0, 0, 1,
0], [0, 0, 0, 1]]):

T1_0 := Matrix([[cos(q1), -sin(q1), 0, 0], [sin(q1), cos(q1), 0, 0], [0, 0, 1, h],
[0, 0, 0, 1]]):
T2_1 := Matrix([[sin(q2), cos(q2), 0, 0], [0, 0, 1, 0], [cos(q2), -sin(q2), 0, 0],
[0, 0, 0, 1]]):
T3_2 := Matrix([[sin(q3), cos(q3), 0, a2], [cos(q3), -sin(q3), 0, 0], [0, 0, -1,
0], [0, 0, 0, 1]]):
T4_3 := Matrix([[cos(q4), -sin(q4), 0, a3], [0, 0, -1, -d4], [sin(q4), cos(q4), 0,
0], [0, 0, 0, 1]]):
T5_4 := Matrix([[cos(q5), -sin(q5), 0, 0], [0, 0, -1, 0], [sin(q5), cos(q5), 0,
0], [0, 0, 0, 1]]):
T6_5 := Matrix([[cos(q6), -sin(q6), 0, 0], [0, 0, 1, d6], [-sin(q6), -cos(q6), 0,
0], [0, 0, 0, 1]]):
T6_0 := ((((T1_0 . T2_1) . T3_2) . T4_3) . T5_4) . T6_5:

eval(T6_0, [q1 = 0, q2 = 0, q3 = 0, q4 = 0, q5 = 0, q6 = 0]);
```

II. Analytical Inverse Kinematics with Maple

To systematically derive the 8 analytical solutions for the robot arm, we can exploit three binary geometric branches. By representing these branches with sign parameters $s_1, s_2, s_3 \in \{-1, 1\}$, we can formulate a single, unified mathematical structure.

The 3 Binary Sign Branches

1. **Shoulder configuration (Left / Right):** Represented by $s_1 \in \{-1, 1\}$. Flipping this sign rotates Joint 1 by 180° and flips the arm backwards.
2. **Elbow configuration (Up / Down):** Represented by $s_2 \in \{-1, 1\}$. This controls the sign of the square root when solving the circle-circle intersection for Joint 2.
3. **Wrist configuration (Flip / Non-flip):** Represented by $s_3 \in \{-1, 1\}$. This controls the sign of the square root when extracting Joint 5, which determines whether the wrist flips or remains upright.

Maple Script

The following Maple script defines the target variables symbolically, computes the 8 permutations using a nested loop, and stores the results in an array. Notice that this calculation may take a few minutes.

```
restart;
with(LinearAlgebra):
with(CodeGeneration):

# Define symbolic input variables for the target pose
# T_target = [ R11, R12, R13, Xt ]
#            [ R21, R22, R23, Yt ]
#            [ R31, R32, R33, Zt ]
#            [   0,   0,   0,  1 ]

# Nominal physical constants
h := 2435/10000;
a2 := 2002/10000;
a3 := 87/1000;
d4 := 22761/100000;
d6 := 625/10000;

# Allocate an array to store the 8 solutions (8 rows, 6 columns)
q_sol := Array(1..8, 1..6):

# Counter for solution index
idx := 0:

# Loop over the 3 binary signs to generate all 8 permutations
for s1 in [-1, 1] do
    for s2 in [-1, 1] do
        for s3 in [-1, 1] do
            idx := idx + 1:

            # 1. Wrist Center computation
            Pwc_x := Xt - d6 * R13:
            Pwc_y := Yt - d6 * R23:
            Pwc_z := Zt - d6 * R33:

            # 2. Joint 1 (Shoulder branch controlled by s1)
            # Maple's arctan(y, x) behaves like standard atan2(y, x)
            q1 := arctan(s1 * Pwc_y, s1 * Pwc_x):

            # 3. Joint 2 (Elbow branch controlled by s2)
            v1_x := Pwc_x * cos(q1) + Pwc_y * sin(q1):
            v1_z := Pwc_z - h:

            K := (v1_x^2 + v1_z^2 + a2^2 - a3^2 - d4^2) / (2 * a2):
            q2 := arctan(K, s2 * sqrt(v1_x^2 + v1_z^2 - K^2)) - arctan(v1_z,
v1_x):

            # 4. Joint 3
            num_cos := a3 * (v1_x - a2 * sin(q2)) - d4 * (v1_z - a2 * cos(q2)):
            num_sin := -d4 * (v1_x - a2 * sin(q2)) - a3 * (v1_z - a2 * cos(q2)):

            # Since both num_cos and num_sin are divided by (a3^2 + d4^2),
            # we can omit the divisor inside arctan as it doesn't change the ratio
```

```
or signs.
            theta23 := arctan(num_sin, num_cos):
            q3 := q2 - theta23:

            # 5. Extracting relative wrist target rotation R_wrist = (R3_0)^T *
R_target
            # These are the specific projection elements needed to extract q4, q5,
q6
            R_prime_13 := cos(q1)*cos(theta23)*R13 + sin(q1)*cos(theta23)*R23 -
sin(theta23)*R33:
            R_prime_23 := cos(q1)*sin(theta23)*R13 + sin(q1)*sin(theta23)*R23 +
cos(theta23)*R33:
            R_prime_33 := sin(q1)*R13 - cos(q1)*R23:

            R_prime_21 := cos(q1)*sin(theta23)*R11 + sin(q1)*sin(theta23)*R21 +
cos(theta23)*R31:
            R_prime_22 := cos(q1)*sin(theta23)*R12 + sin(q1)*sin(theta23)*R22 +
cos(theta23)*R32:

            # 6. Wrist Joints (Wrist branch controlled by s3)
            # Utilizing s3 mathematically replaces the need for signum() or sgn()
functions,
            # keeping the equations purely algebraic and continuous.
            q5 := arctan(s3 * sqrt(1 - R_prime_23^2), -R_prime_23):
            q4 := arctan(-R_prime_33 * s3, -R_prime_13 * s3):
            q6 := arctan(R_prime_22 * s3, -R_prime_21 * s3):

            # Store the current solution set
            q_sol[idx, 1] := q1:
            q_sol[idx, 2] := q2:
            q_sol[idx, 3] := q3:
            q_sol[idx, 4] := q4:
            q_sol[idx, 5] := q5:
            q_sol[idx, 6] := q6:
        end do:
    end do:
end do:
```

Code Generation

Once the array is populated, we generate C code using:

```
CodeGeneration[C](q_sol, optimize=true, deducetypes=false, defaulttype=double);
```

III. Jacobian Matrix Determinant

For classical 6-axis robotic arms, we have:

$$\det(J) = \det(J_{\text{translation}}) \cdot \det(J_{\text{rotation}})$$

which is known as Singularity Decoupling.

The Decoupled $\det(J)$ Equation

By solving the determinants of the 3×3 sub-blocks analytically, the full determinant of the robot's Jacobian evaluates to:

$$\det(J) = a_2 \cdot v_{1,x} \cdot (a_3 \cos(q_3) + d_4 \sin(q_3)) \cdot \cos(q_4) \sin(q_5)$$

Where:

$$v_{1,x} = a_2 \sin(q_2) + a_3 \cos(q_2 - q_3) - d_4 \sin(q_2 - q_3)$$

$$a_2 = 0.2002$$

$$a_3 = 0.087$$

$$d_4 = 0.22761$$

Physical Interpretation of the Factors:

1. **Shoulder Singularity ($v_{1,x} = 0$):** This occurs when the wrist center lies directly on the rotational Z-axis of Joint 1. Joint 1 loses the ability to produce lateral Cartesian motion.
2. **Elbow Singularity ($a_3 \cos(q_3) + d_4 \sin(q_3) = 0$):** This happens when the elbow is fully extended or fully folded.
3. **Planar Projection Singularity ($\cos(q_4) = 0$):** At $q_4 = \pm 90^\circ$, the rotational axes of Joints 5 and 6 lie in the same physical plane, locking a degree of freedom.
4. **Wrist Singularity ($\sin(q_5) = 0$):** At $q_5 = 0$ (the robot's home alignment), the rotational axes of Joints 4 and 6 become collinear, causing "wrist-lock".

Maple Code Generation Script

```
restart;
with(CodeGeneration):

# Physical dimensions of the robot from URDF
a2 := 2002/10000;
a3 := 87/1000;
d4 := 22761/100000;

# 1. Translational radius (Shoulder boundary)
v1_x := a2 * sin(q2) + a3 * cos(q2 - q3) - d4 * sin(q2 - q3);

# 2. Elbow boundary term
elbow_term := a3 * cos(q3) + d4 * sin(q3);

# 3. Wrist boundary term (Unique orthogonal design)
wrist_term := cos(q4) * sin(q5);

# 4. Total Decoupled Jacobian Determinant
det_J := a2 * v1_x * elbow_term * wrist_term;
```

```
# Generate the C Code
CodeGeneration[C](det_J, optimize=true, defaulttype=double);
```